임베디드 시스템

Embedded Systems

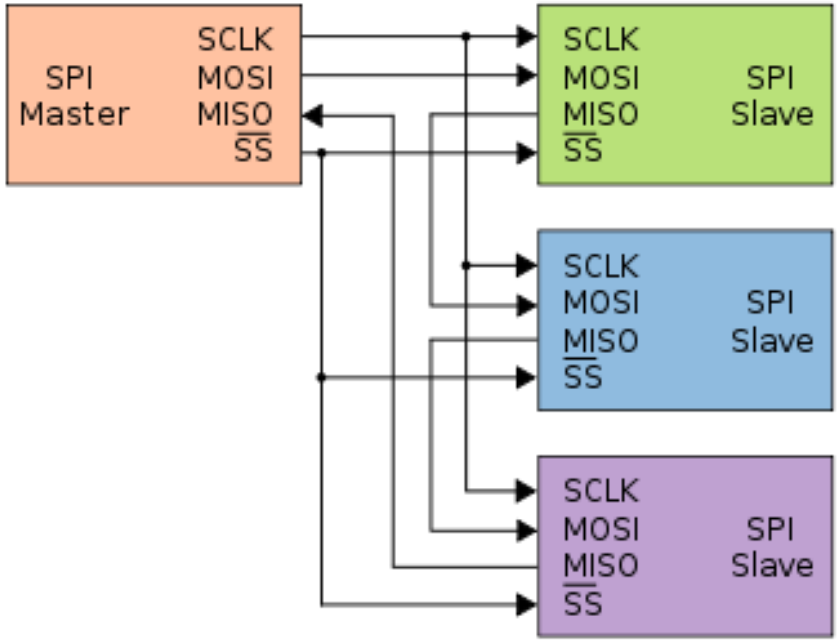
Potentiometer | CDS | RFID

강원대학교 컴퓨터공학과 권용훈

SPI (1/2)

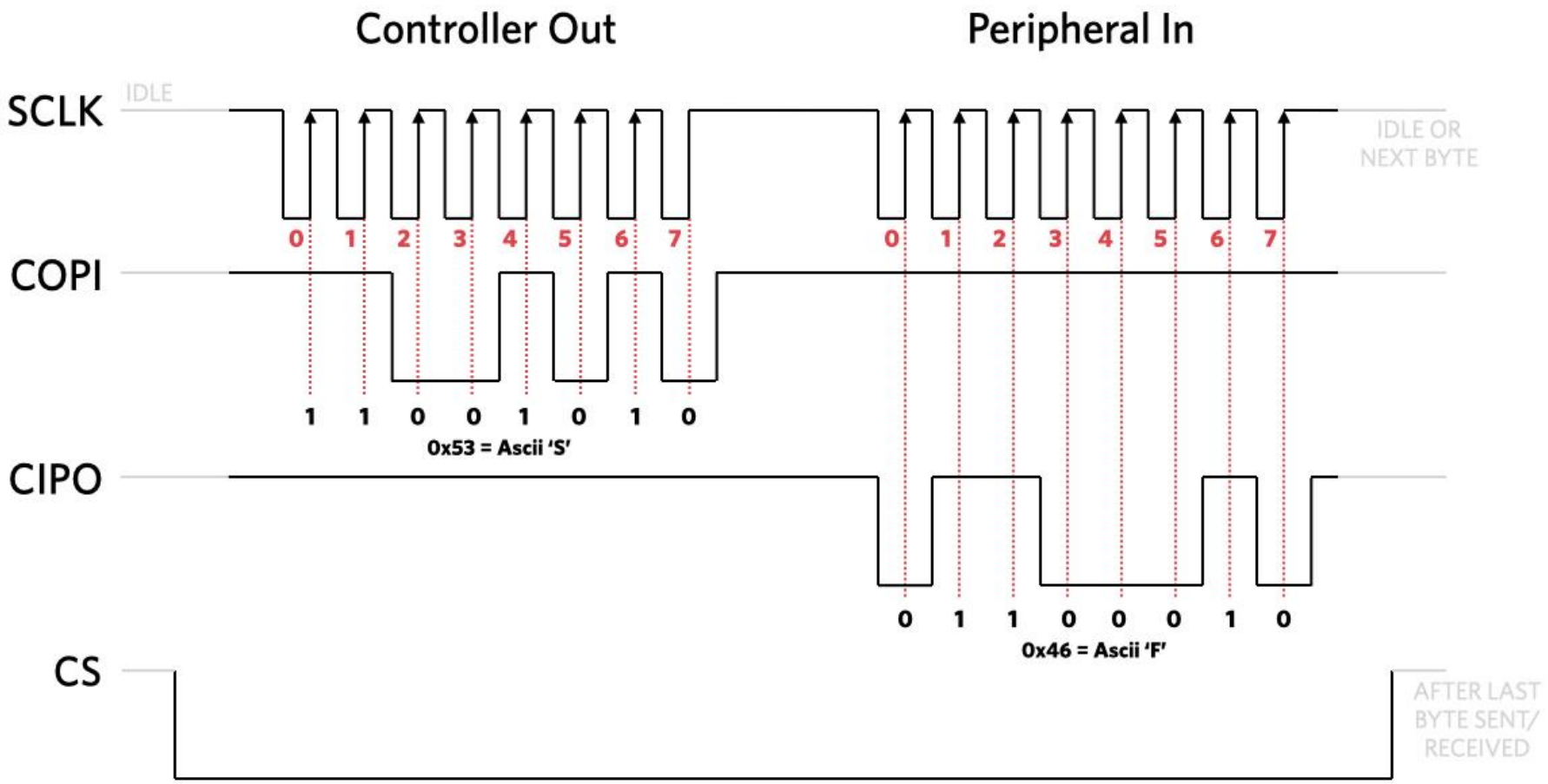
SPI(Serial Peripheral Interface)

- 시리얼 통신의 일종으로 주변 기기와의 데이터 교환을 위한 방식
- 전이중 통신이며 동기식 방식
- I2C 방식보다는 빠르나 마스터 장치는 하나만 사용할 수 있다.
- SCLK(SCL): Clock 신호를 발생시키는 핀
- MOSI(Master Output Slave Input): Master -> Slave
- MISO(Master Input Slave Output): Slave -> Master
- SS(Slave Select): Slave를 선택하기 위한 핀
    - I2C에서 사용하는 slave address를 사용하지 않음



SPI (2/2)

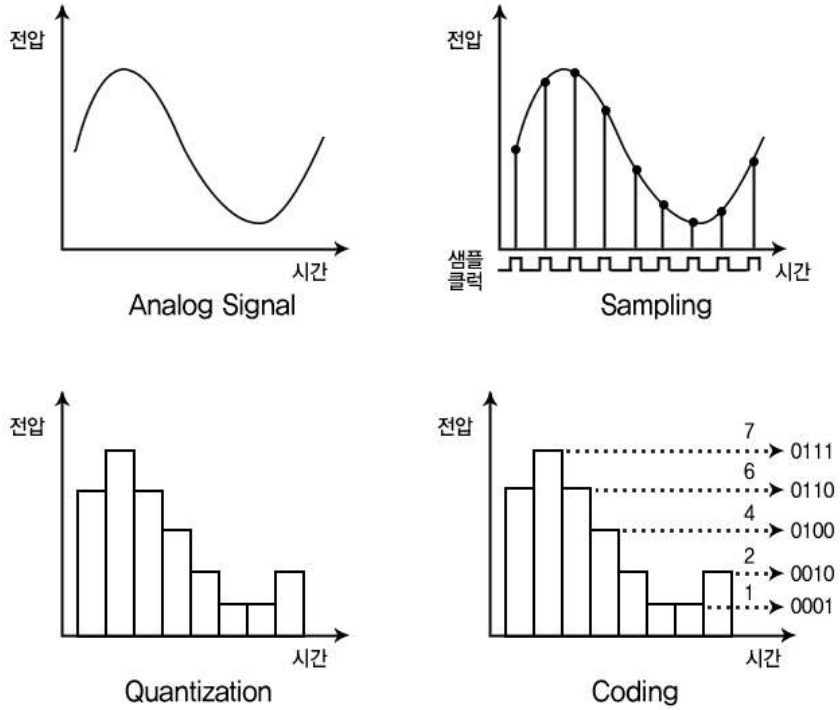
SPI 통신 원리



Potentiometer(1/4)

ADC(Analog-to-Digital Converter)

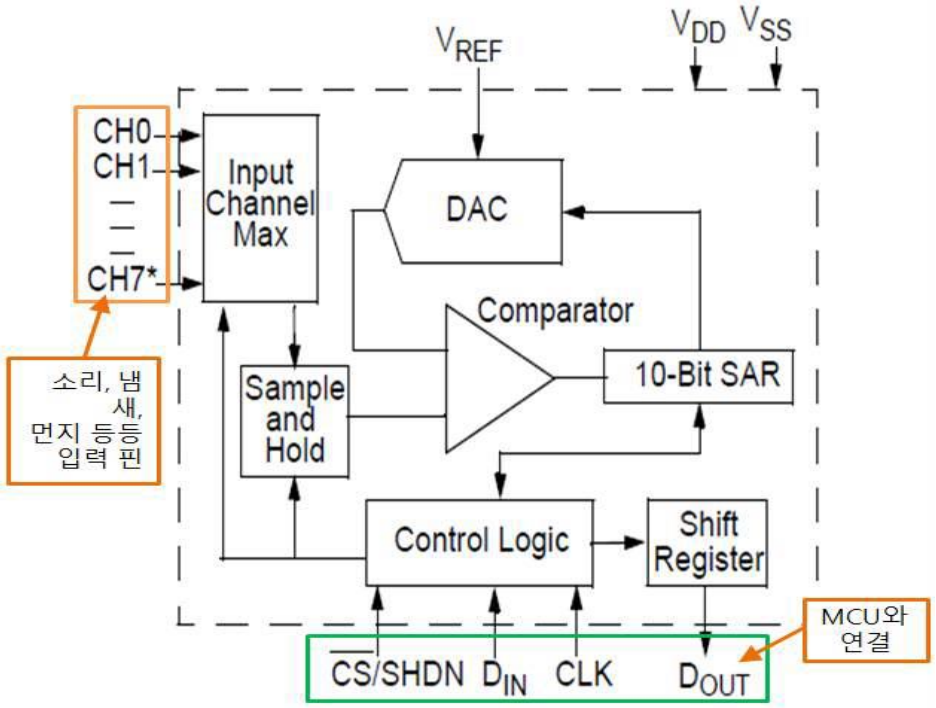
- 연속적인 값을 가지는 아날로그 신호를 디지털 신호로 변환하는 장치
- 다음과 같은 절차를 통해 변환된다.
- 표본화(Sampling)
  - 아날로그 입력 시 일정 시간 간격으로 표본을 추출한 것
  - 일정 시간 간격이 짧을수록 아날로그 입력 신호에 근접한 모양의 표본 추출 가능
  - 단위 : SPS (Sample Per Second / Sampling Rate)
- 양자화(Quantization)
  - 시간에 따른 신호의 세기를 이산화 시켜 표시하는 것
  - 이산화 값의 범위가 클수록 정밀도가 높으며 이것을 해상도(Resolution)이라 한다.
  - 8bit (0~ 255), 10bit (0 ~ 1023)
- 부호화(Coding)
  - 표본화 된 데이터를 디지털 값으로 맵핑(mapping)하여 사용하는 것

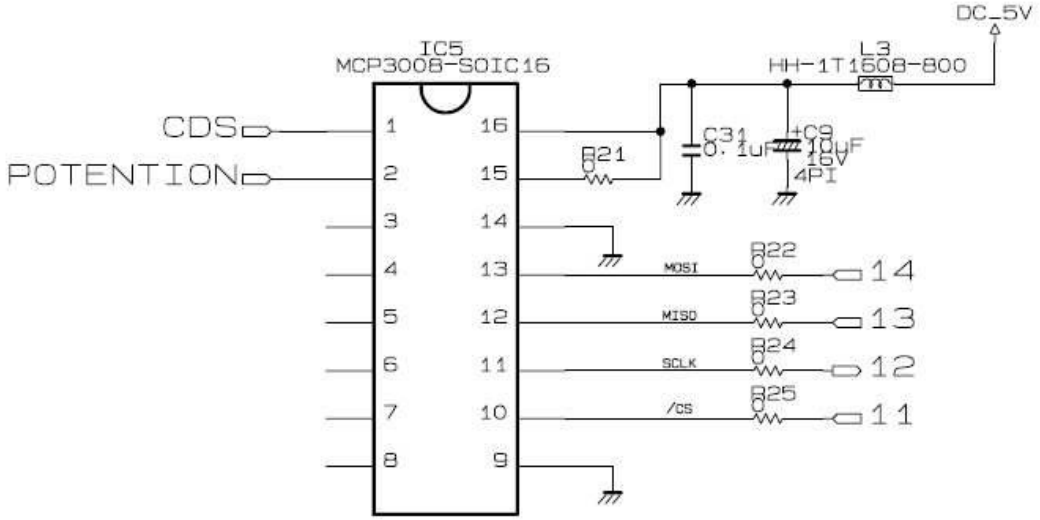


Potentiometer(2/4)

MCP3008 IC

- 아날로그 데이터를 SPI방식으로 입력 받는 집적회로

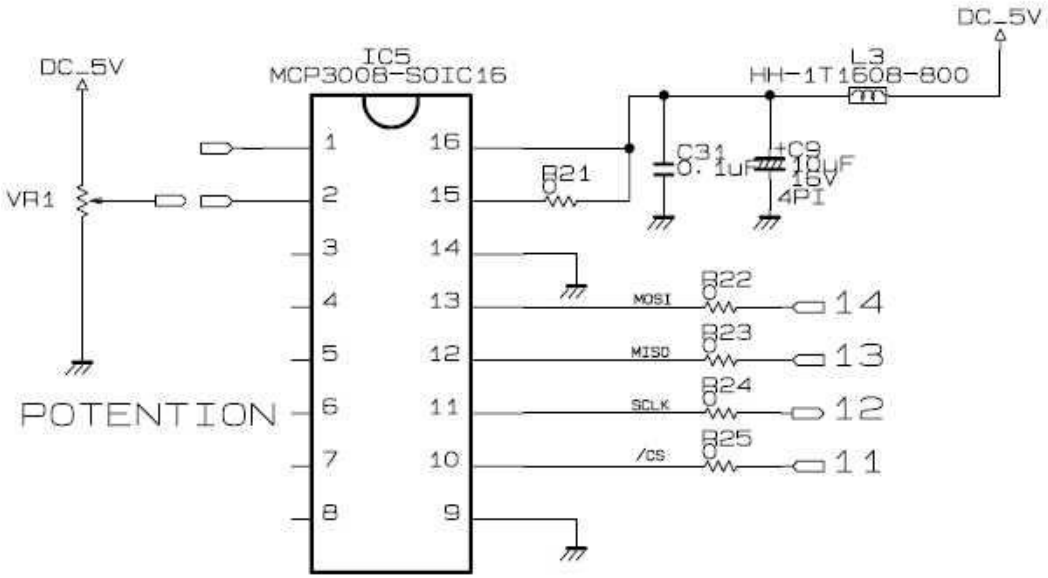




Potentiometer(3/4)

가변저항(Potentiometer)

- 전자회로의 저항 값을 정해진 범위 내에서 임의로 바꿀 수 있는 저항



		3.3v			17	18	0	IN	GPIO. 5	5	24
10	12	MOSI	ALT0	0	19	20			0v		
9	13	MISO	ALT0	0	21	22	0	IN	GPIO. 6	6	25
11	14	SCLK	ALT0	0	23	24	1	OUT	CE0	10	8
		0v			25	26	1	OUT	CE1	11	7
0	30	SDA.0	IN	0	27	28	0	IN	SCL.0	31	1
5	21	GPIO.21	OUT	0	29	30			0v		
6	22	GPIO.22	IN	0	31	32	0	IN	GPIO.26	26	12
13	23	GPIO.23	IN	0	33	34			0v		
19	24	GPIO.24	IN	0	35	36	0	OUT	GPIO.27	27	16
26	25	GPIO.25	IN	0	37	38	0	IN	GPIO.28	28	20
		0v			39	40	0	IN	GPIO.29	29	21
BCM	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	BCM	

Pi 3B+

Potentiometer(4/4)

```c
#include <stdio.h>
#include <wiringPi.h>
#include <wiringPiSPI.h>
#include <errno.h>
#include <string.h>
#define CHAN_CONFIG_SINGLE 8 //- 싱글 채널 설정 값
#define SPI_SPEED 1000000 //- 1MHz
#define SPI_CE_CHANNEL 1 //- SPI와 연결된 CE 채널
#define SPI_ADC_POTEN_CHANNEL 1 //- Potentionmeter 연결 채널
#define CS_MCP3208 11 //- MCP3208 연결 핀
int spiSetup(void);
int readAnalogData(int adcChannel);
int main(void){
      if( wiringPiSetup() == -1){
        printf("Setup Fail ~!!");
        return -1;
    }
    //- SPI통신 초기화
    spiSetup();
    //- ADC IC MCP3008 초기화
    pinMode(CS_MCP3208, OUTPUT);
    int adcValue = 0;
      while(1){
        //- 센서 값 읽기
        adcValue = readAnalogData(SPI_ADC_POTEN_CHANNEL);
        printf(" Potentionmeter Value = %d\n", adcValue);
        delay(500);
    }
    return 0;
}
```

```c
int spiSetup(void){
    if( wiringPiSPISetup (SPI_CE_CHANNEL, SPI_SPEED) == -1 ){
        fprintf (stderr, "wiringPiSPISetup Failed ! ERROR : %s\n",
strerror (errno));
        return 1;
    }
}
//- 채널 1번에서 측정된 ADC센서 읽어오기 ----------------------------
----------------
int readAnalogData(int adcChannel){
    int adcValue = 0;
    unsigned char buffer[3] = {0};
    //- MCP3008의 8개 채널 중 하나인지 체크
    if(adcChannel<0 || adcChannel>7) return -1;
    //- SPI 통신 버퍼 설정
    buffer[0] = 0x01;
    buffer[1] = (CHAN_CONFIG_SINGLE+adcChannel) << 4;
    //- SPI 통신할 칩 활성화
    digitalWrite(CS_MCP3208, LOW);
    //- 채널 1에서 측정된 ADC 센서 읽어 오기
    wiringPiSPIDataRW(SPI_CE_CHANNEL, buffer, 3);
    //- 읽어온 값에서 10비트 데이터 추출하기
    adcValue = ( (buffer[1] & 3 ) << 8 ) + buffer[2];
    //- SPI 통신할 칩 비활성화
    digitalWrite(CS_MCP3208, HIGH);
    return adcValue;
}
```

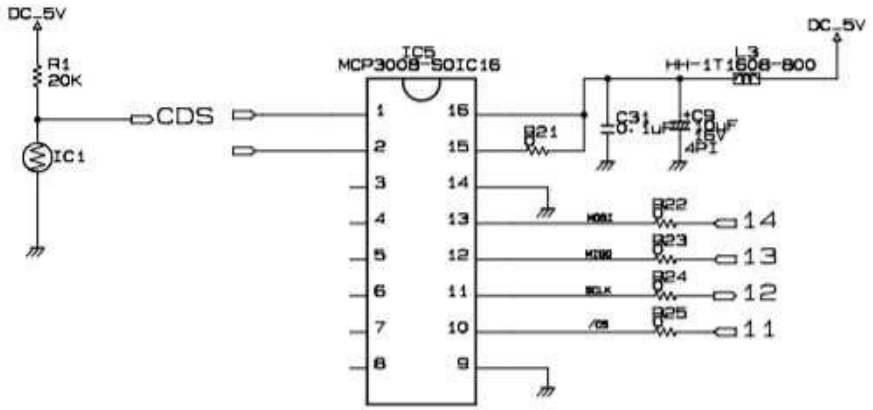
CDS (1/2)

조도(Light)센서

- 광전도효과를 이용하여 주위의 빛의 양을 측정하여 출력해주는 센서
- CDS(Cadmium sulfide): 황화 카드뮴
- 빛이 센서에 조사되면 광전도효과에 의해 내부저항이 감소한다.
- 빛의 세기와 저항 값은 반비례 관계가 성립한다.

		3.3v			17	18	0	IN	GPIO. 5	5	24
10	12	MOSI	ALT0	0	19	20			0v		
9	13	MISO	ALT0	0	21	22	0	IN	GPIO. 6	6	25
11	14	SCLK	ALT0	0	23	24	1	OUT	CE0	10	8
		0v			25	26	1	OUT	CE1	11	7
0	30	SDA.0	IN	0	27	28	0	IN	SCL.0	31	1
5	21	GPIO.21	OUT	0	29	30			0v		
6	22	GPIO.22	IN	0	31	32	0	IN	GPIO.26	26	12
13	23	GPIO.23	IN	0	33	34			0v		
19	24	GPIO.24	IN	0	35	36	0	OUT	GPIO.27	27	16
26	25	GPIO.25	IN	0	37	38	0	IN	GPIO.28	28	20
		0v			39	40	0	IN	GPIO.29	29	21

BCM	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	BCM
					Pi 3B					



CDS (2/2)

```c
#include <stdio.h>
#include <wiringPi.h>
#include <wiringPiSPI.h>
#include <errno.h>
#include <string.h>
#define CHAN_CONFIG_SINGLE 8 //- 싱글 채널 설정 값
#define SPI_SPEED 1000000 //- 1MHz
#define SPI_CE_CHANNEL 1 //- SPI와 연결된 CE 채널
#define SPI_ADC_LIGHT_CHANNEL 0 //- Light 연결 채널
#define CS_MCP3208 11 //- MCP3208 연결 핀
int spiSetup(void);
int readAnalogData(int adcChannel);
int main(void){
    //- wiringPi 초기화
    if( wiringPiSetup() == -1){
        printf("Setup Fail ~!!");
        return -1;
    }
    //- SPI통신 초기화
    spiSetup();
    //- ADC IC MCP3008 초기화
    pinMode(CS_MCP3208, OUTPUT);
    int adcValue = 0;
    //- 센서 읽어서 출력
    while(1){
    //- 센서 값 읽기
    adcValue = readAnalogData(SPI_ADC_LIGHT_CHANNEL);
    printf(" Light Value = %dLux\n", adcValue);
    delay(500);
    }
    return 0;
}
```

```c
int spiSetup(void){
    if( wiringPiSPISetup (SPI_CE_CHANNEL, SPI_SPEED) == -1 ){
        fprintf (stderr, "wiringPiSPISetup Failed ! ERROR : %s\n",
strerror (errno));
        return 1;
    }
}
//- 채널 1번에서 측정된 ADC센서 읽어오기 ----------------------------
----------------
int readAnalogData(int adcChannel){
    int adcValue = 0;
    unsigned char buffer[3] = {0};
    //- MCP3008의 8개 채널 중 하나인지 체크
    if(adcChannel<0 || adcChannel>7) return -1;
    //- SPI 통신 버퍼 설정
    buffer[0] = 0x01;
    buffer[1] = (CHAN_CONFIG_SINGLE+adcChannel) << 4;
    //- SPI 통신할 칩 활성화
    digitalWrite(CS_MCP3208, LOW);
    //- 채널 1에서 측정된 ADC 센서 읽어 오기
    wiringPiSPIDataRW(SPI_CE_CHANNEL, buffer, 3);
    //- 읽어온 값에서 10비트 데이터 추출하기
    adcValue = ( (buffer[1] & 3 ) << 8 ) + buffer[2];
    //- SPI 통신할 칩 비활성화
    digitalWrite(CS_MCP3208, HIGH);
    return adcValue;
}
```

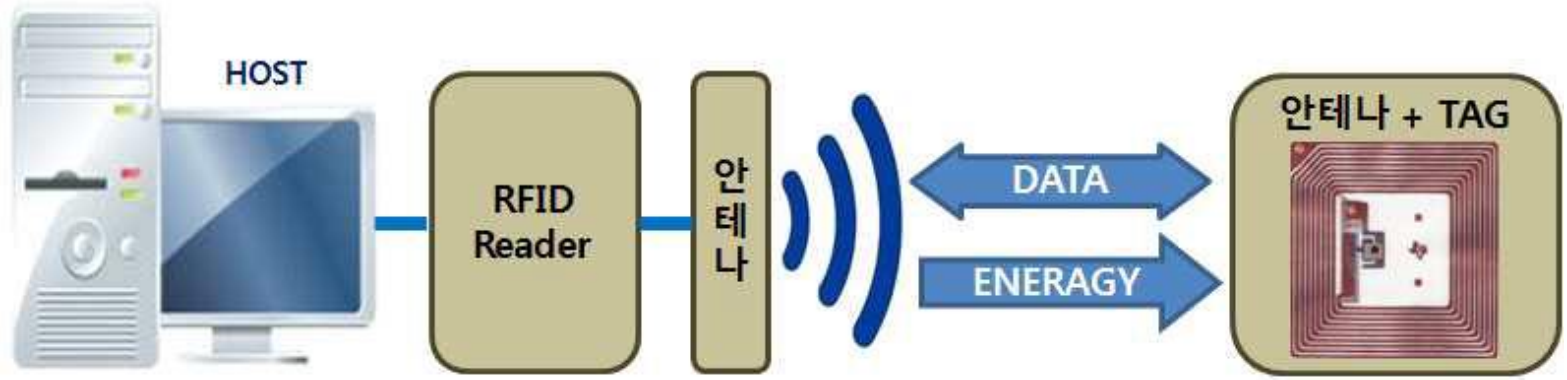
RFID (1/8)

RFID(Radio-Frequency Identification)

- 전파를 이용해 먼 거리에서 IC 칩의 정보를 인식하는 것
- 전자 태그, 스마트 태그, 전자 라벨 등에 활용
- 바코드 방식보다 비교적 먼 거리를 인식할 수 있고 장애물을 통과할 수 있는 장점이 있다.
- 보안이 취약하고 같은 주파수를 읽는 판독기가 있으면 쉽게 역추적 할 수 있다.
- 국제 규격이 아직 없어 제조사 마다 주파수가 다르다.

RFID (2/8)

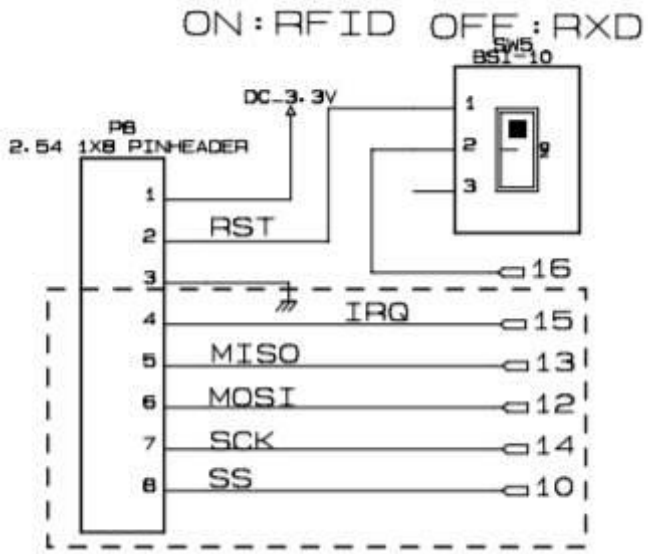
RFID 통신 구성



RFID Tag	- 안테나 + 칩 + 정보 - 제품이나 대상에 부착 및 삽입 - 종류 : 능동형(내부 전원 보유) /수동형(리더기 전원사용) - 형태 : 라벨형, 카드형, 밴드형, 메탈태그, 원형태그 등등
안테나	- 태그로부터 전송된 데이터를 수신하여 리더로 전달 - 태그 크기, 거리, 환경에 따라 다양한 크기
RFID Reader (판독기)	- Tag 정보 송수신 기능 + 수집된 정보를 네트워크로 전송 - 형태 : 고정형, 이동형, 휴대형
호스트	- 리더로부터 수신된 데이터 처리 및 활용

RFID (3/8)

RFID 통신 모듈



IC	NXP사의 MFRC522
지원 버전	- ISO/IEC 14443 A/MIFARE
동작 주파수	- **13.56MHz**
인터페이스 방식	- SPI 방식 / UART / I2CV
전송속도	- 최대 10Mbit/s
구동전원	- 3.3V
지원 카드 타입	- Mifare 1 S70 MIFARE Ultralight - mifare Pro - MIFARE DESFire

SDA	- SPI 통신용 핀
SCK	
MOSI	
MISO	
IRQ	- NC
GND	- GND 연결
RST	- Reset
3.3V	- 3.3V 전원 연결

RFID (4/8)

RFID 태그 구조

16 Bytes

4 Blocks

16 Sectors

Read-Only

Sector	Block	Byte Number within a Block 0 1 2 3 4 5	6 7 8 9	10 11 12 13 14 15	Description
15	3	Key A	Access Bits	Key B	Sector Trailer 15
	2				Data
	1				Data
	0				Data
14	3	Key A	Access Bits	Key B	Sector Trailer 14
	2				Data
	1				Data
	0				Data
:	:				
:	:				
:	:				
1	3	Key A	Access Bits	Key B	Sector Trailer 1
	2				Data
	1				Data
	0				Data
0	3	Key A	Access Bits	Key B	Sector Trailer 0
	2				Data
	1				Data
	0	Manufacturer Data			Manufacturer Block

Key

Sector 15: 8fd0a4f256e9
Sector 14: 533cb6c723f6
Sector 13: a0478cc39091
Sector 12: 587ee5f9350f
Sector 11: 714c5c886e97
Sector 10: aabbccddeeff
Sector 9: d3f7d3f7d3f7
Sector 8: aabbccddeeff
Sector 7: 000000000000
Sector 6: 1a982c7e459a
Sector 5: 4d3a99c351dd
Sector 4: b0b1b2b3b4b5
Sector 3: a0a1a2a3a4a5
Sector 2: a1b1c1d1e1f1
Sector 1: a0b0c0d0e0f0
Sector 0: ffffffffffff

RFID (5/8)

RFID 라이브러리 함수

char PcdReset(void)		
위 치	MFRC522.h	
기 능	초기화	
매개변수	void	없음
리턴 값	char	MI_OK

void PcdAntennaOn(void)		
위 치	MFRC522.h	
기 능	RFID Reader기 내부 안테나 ON	
매개변수	void	없음
리턴 값	void	없음

void PcdAntennaOff(void)		
위 치	MFRC522.h	
기 능	RFID Reader기 내부 안테나 OFF	
매개변수	void	없음
리턴 값	void	없음

char PcdHalt(void)		
위 치	MFRC522.h	
기 능	RFID Reader 동작 종료	
매개변수	void	없음
리턴 값	char	MI_OK / MI_NOTAGERR/ MI_ERR

char PcdRequest(unsigned char req_code,unsigned char *pTagType)		
위 치	MFRC522.h	
기 능	RFID 태그 타입 읽어오기	
매개변수	unsigned char req_code	요청 코드
	unsigned char *pTagType	RFID 태그 타입 0x4400 = Mifare_UltraLight 0x0400 = Mifare_One(S50) 0x0200 = Mifare_One(S70) 0x0800 = Mifare_Pro(X) 0x4403 = Mifare_DESFire
리턴 값	char	MI_OK / MI_ERR

RFID (6/8)

RFID 라이브러리 함수

char PcdAnticoll(unsigned char *pSnr)		
위 치	MFRC522.h	
기 능	태그 간의 충돌을 방지하는 함수	
매개변수	unsigned char *pSnr	
리턴 값	char	MI_OK / MI_ERR

char PcdSelect(unsigned char *pSnr)		
위 치	MFRC522.h	
기 능	태그의 정보를 가져오는 함수	
매개변수	unsigned char *pSnr	
리턴 값	char	MI_OK / MI_ERR

char PcdAuthState(unsigned char auth_mode,unsigned char addr, unsigned char *pKey,unsigned char *pSnr)		
위 치	MFRC522.h	
기 능	인증 받은 카드 여부 체크	
매개변수	unsigned char auth_mode	인증 모드
	unsigned char addr	메모리 주소
	unsigned char *pKey	인증검사 기본키 값
	unsigned char *pSnr	검사 받을 키 값
리턴 값	char	MI_OK / MI_ERR

char PcdRead(unsigned char addr,unsigned char *pData)		
위 치	MFRC522.h	
기 능	데이터 읽어오기	
매개변수	unsigned char addr	읽기 시작할 메모리 주소
	unsigned char *pData	읽은 데이터
리턴 값	char	MI_OK / MI_ERR

char PcdWrite(unsigned char addr,unsigned char *pData)		
위 치	MFRC522.h	
기 능	데이터 쓰기	
매개변수	unsigned char addr	쓰기 시작할 메모리 주소
	unsigned char *pData	쓸 데이터
리턴 값	char	MI_OK / MI_ERR

RFID (7/8)

RFID 라이브러리 함수

char PcdBakValue(unsigned char sourceaddr, unsigned char goaladdr)		
위 치	MFRC522.h	
기 능	블록의 데이터를 복사	
매개변수	unsigned char sourceaddr	복사할 데이터의 주소
	unsigned char goaladdr	저장할 데이터의 주소
리턴 값	char	MI_OK / MI_ERR

```c
#include <errno.h>
#include <string.h>
#include "MFRC522.h"

#define CHAN_CONFIG_SINGLE 8 //- 싱글 채널 설정 값
#define SPI_SPEED 1000000 //- 1MHz
#define SPI_CE0_CHANNEL 0 //- SPI와 연결된 CE0 채널

int main (void){
    unsigned char data1[16] = {0x12, 0x34, 0x56, 0x78, 0xED, 0xCB,
    0xA9, 0x87, 0x12, 0x34, 0x56,
    0x78, 0x01, 0xFE, 0x01, 0xFE};
    unsigned char DefaultKey[6] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
    unsigned char g_ucTempbuf[20];
    int count = 0;
    unsigned char status, i;
    //- wiringPi 초기화
    if( wiringPiSetup() == -1){
        printf("Setup Fail ~!!");
        return -1;
    }

    if( wiringPiSPISetup (SPI_CE0_CHANNEL, SPI_SPEED) == -1 ){
        fprintf (stderr, "wiringPiSPISetup Failed ! ERROR : %s\n",
strerror (errno));
        return 1;
    }
    delay(500);
    PcdReset(); PcdAntennaOff();
    delay(10);
    PcdAntennaOn();
```

RFID (8/8)

```c
printf ("... Touch RFID CARD\n\n") ;
    while(1){
        status = PcdRequest(PICC_REQALL, g_ucTempbuf);
        if (status != MI_OK) continue;
        printf("\ncard type:");
        switch(g_ucTempbuf[0]){
            case 0x02: printf("Mifare_One(S70)\n"); break;
            case 0x04: printf("Mifare_One(S50)\n"); break;
            case 0x08: printf("Mifare_Pro(X)\n"); break;
            case 0x44:{
                if(g_ucTempbuf[1] == 0x00){
                    printf("Mifare_UltraLight\n");
                    break;
                }else if(g_ucTempbuf[1] == 0x03){
                    printf("Mifare_DESFire\n");
                    break;
                }
            }
        }
        status = PcdAnticoll(g_ucTempbuf);
        if (status != MI_OK) continue;
        printf("\ncard SN:");
        for(i=0;i<4;i++){
            if(g_ucTempbuf[i] < 0x10)
                printf("0%X",g_ucTempbuf[i]);
            else
                printf("%X",g_ucTempbuf[i]);
        }
        printf("\n");
        status = PcdSelect(g_ucTempbuf);
        if (status != MI_OK) continue;
        status = PcdAuthState(PICC_AUTHENT1A, 1, DefaultKey, g_ucTempbuf);
        if (status != MI_OK) continue;
        status = PcdWrite(1, data1);
        if (status != MI_OK) continue;
        status = PcdBakValue(1, 2);
        for(i=0;i<4;i++){
            status = PcdRead(i, g_ucTempbuf);
            if (status != MI_OK) continue;
            printf("\nread block %d:", i);
            for(int j=0;j<16;j++){
                if(g_ucTempbuf[j] < 0x10)
                    printf("0%X",g_ucTempbuf[j]);
                else
                    printf("%X",g_ucTempbuf[j]);
            }
        }
        printf("\n");
        PcdHalt();
        delay(1000);
        if(count++>=3) break;
    }
    return 0;
}
```

컴파일

ex) gcc -o [**실행파일명**] [**코드파일**] MFRC522.c -lwiringPi

과제 11

가변저항과 조도센서를 이용한 금고를 만든다.

- 초기에는 아무 일도 일어나지 않는다.
- enter 키를 누르면 시스템이 시작되고, 10~1000 사이의 정수형 난수를 생성하여 키 값으로 저장한다.
- 가변 저항의 값이 화면상에 주기적으로 출력된다.
- 가변 저항의 값이 키 값의 ± 100 범위에 들어오면 빨간색 LED가 켜진다.
- 가변 저항의 값이 키 값의 ± 10 범위에 들어오면 수동부저로 상승 음계를 출력한다. 이 때 금고의 문이 열린다.
- 금고의 문이 열렸을 때, 조도가 정상이면 아무일도 일어나지 않는다.
- 조도가 낮으면(사람의 손으로 가렸을 때 도달하는 값) 카메라를 이용하여 사진을 촬영한다.
- 모든 작업이 수행되면 프로그램이 종료된다.

과제 12

가변저항을 이용하여 전자 하모니카를 만드시오. (23-2 중간고사 기출문제)

- 초기에는 어떠한 동작을 수행하지 않고, enter를 누르면 시스템이 시작된다.
- 가변 저항의 값은 수동부저의 주파수를 나타낸다. 가변 저항이 왼쪽 끝에 있을 때 주파수는 대략 200이고, 오른쪽 끝에 있을 때는 주파수가 대략 700이다.
- 키보드 Q를 누르면 해당 주파수의 음을 수동 부저를 통해 0.2초 동안 출력한다.
- 키보드 W를 누르면 해당 주파수의 음을 멈추지 않고 출력한다.
- 부저가 울릴 때는 빨간색 LED가 켜진다.
- 키보드 R을 누르면 수동 부저가 정지하고 LED가 꺼진다.

과제 13

RFID를 사용한 교통카드

- RFID의 0~3번 블록 중에 1번 블록만 사용한다.
- 프로그램이 실행되면 3가지 기능이 수행되며, 기능 선택을 대기한다. 기능은 아래와 같다.
  - recharge : 입력 받은 금액만큼 토큰에 충전하고, 총 금액을 표시한다.
  - payment : 입력 받은 금액만큼 토큰에서 제거하고, 총 금액을 표시한다.
    만약, 입력 받은 금액이 잔액보다 크다면 충분하지 않음을 표시하고, 결제가 취소된다.
  - check: 토큰에 잔액을 표시한다. 처음 check할 경우 금액을 10,000으로 초기화 한다.
- 기능의 수행은 숫자를 입력(getch() 사용)한 다음 토큰이 센서에 접근했을 때 실행된다.
- 기능 외의 숫자를 입력하거나 태그 중에 오류가 발생하면 처음으로 돌아간다.
- 모든 금액은 토큰에 입력되야 한다.
- (권장사항) 블록을 하나의 문자열로 사용한다.

```
pi@raspberrypi:~ $ ./test
... Touch RFID CARD

Select Functions! (1.recharge  2.payment  3.check)
Function 3 is selected
balance: 10000

Select Functions! (1.recharge  2.payment  3.check)
Function 2 is selected
Input the payment amount
2000
balance: 8000

Select Functions! (1.recharge  2.payment  3.check)
Function 3 is selected
balance: 8000

Select Functions! (1.recharge  2.payment  3.check)
Function 1 is selected
Input the recharge amount
5000
balance: 13000

Select Functions! (1.recharge  2.payment  3.check)
Function 2 is selected
Input the payment amount
200000
Not Enough balance
balance: 13000

Select Functions! (1.recharge  2.payment  3.check)
Function 1 is selected
Input the recharge amount
2000
Connection Error!  연결 오류
Select Functions! (1.recharge  2.payment  3.check)
Wrong Function     기능 입력 오류
Select Functions! (1.recharge  2.payment  3.check)
Function 1 is selected
Input the recharge amount
3000
balance: 16000
```

고생하셨습니다

Thank you!